Complexity

Liu Haiyang

haiyangliu@bjtu.edu.cn

Beijing JiaoTong University

A General Portable Performance Metric

- *Informally,* Time to solve a problem of size, n,

  $T(n)$ is $O(\log n)$

  $T(n) = c \log_2 n$

- *Formally:*
  - $O(\ g(n)\ )$ is the set of functions, f, such that

    $f(n) < c\, g(n)$

    for some constant, $c > 0$, and $n > n_0$

    ie for sufficiently large $\boldsymbol{n}$
  - Alternatively, we may write

    $$\lim_{n \to \infty} \frac{f(n)}{g(n)} \leq c$$

    and say

g is an upper bound for f

A General Portable Performance Metric

- $O(g)$
  - the set of functions that grow no faster than g.
- $g(n)$ describes the worst case behaviour of an algorithm that is $O(g)$

A General Portable Performance Metric

- $O(g)$
  - the set of functions that grow no faster than g.
- $g(n)$ describes the worst case behaviour of an algorithm that is $O(g)$
- Two additional notations
- $\Omega(g)$
  - the set of functions, f, such that

$$f(n) > c\,g(n)$$

for some constant, c, and $n > n_0$

g is a lower bound for f

A General Portable Performance Metric

- $O(g)$
  - the set of functions that grow no faster than g.
- $g(n)$ describes the worst case behaviour of an algorithm that is $O(g)$
- Two additional notations
- $\Omega(g)$
  - the set of functions, f, such that

$$f(n) > c\,g(n)$$

for some constant, c, and $n > n_0$

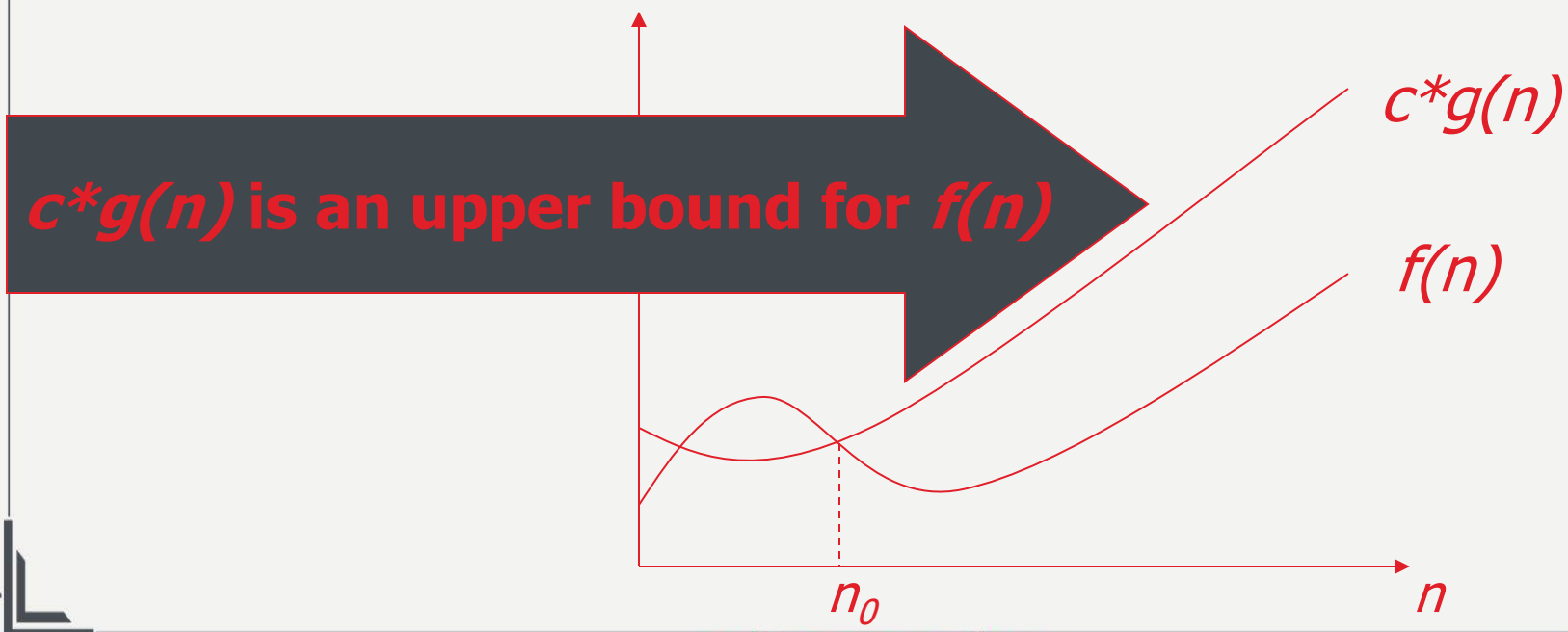
g is a lower bound for f

- $\Theta(g) = O(g) \cap \Omega(g)$

Set of functions growing at the same rate as g

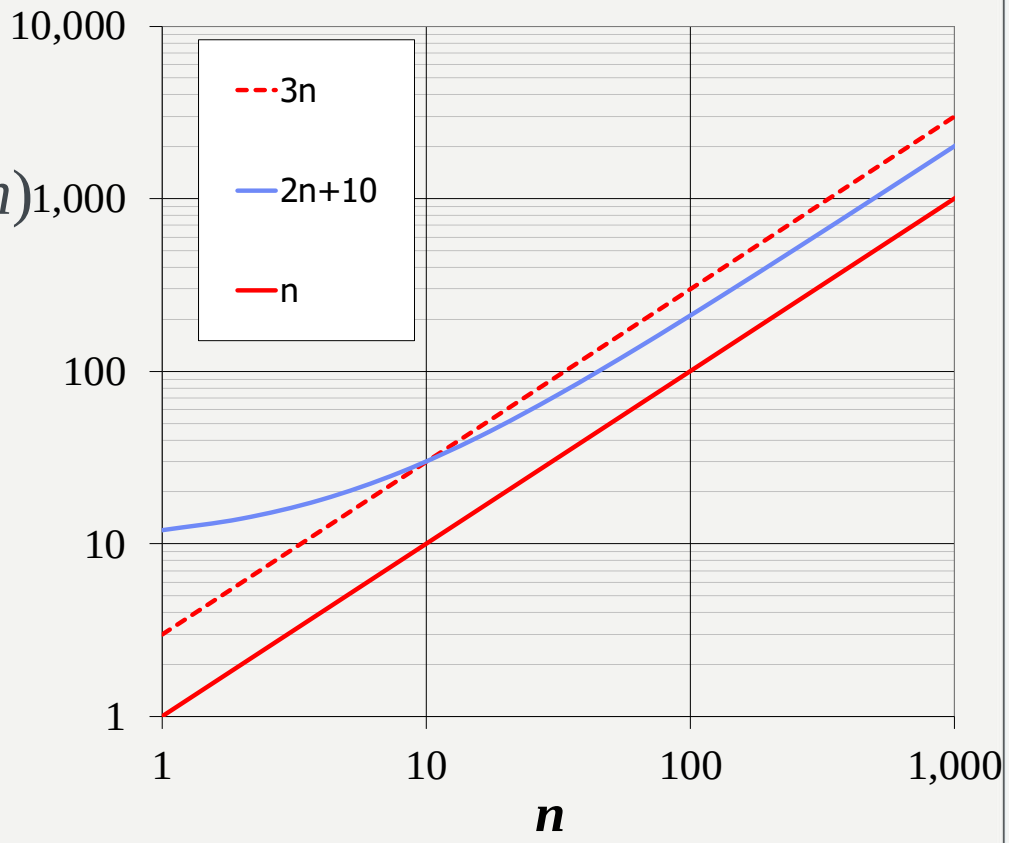
Big-O Defined

- The O symbol was introduced in 1927 to indicate relative growth of two functions based on asymptotic behavior of the functions now used to classify functions and families of functions
- $f(n) = O(g(n))$ *if there are constants* c *and* n_0 *such that* $f(n) < c * g(n)$ *when* $n \geq n_0$



Big-O Example

- Example: $2n + 10$ is $O(n)$
  - $2n + 10 \leq cn$
  - $(c - 2)\, n \geq 10$
  - $n \geq 10/(c - 2)$
  - Pick $c = 3$ and $n_0 = 10$

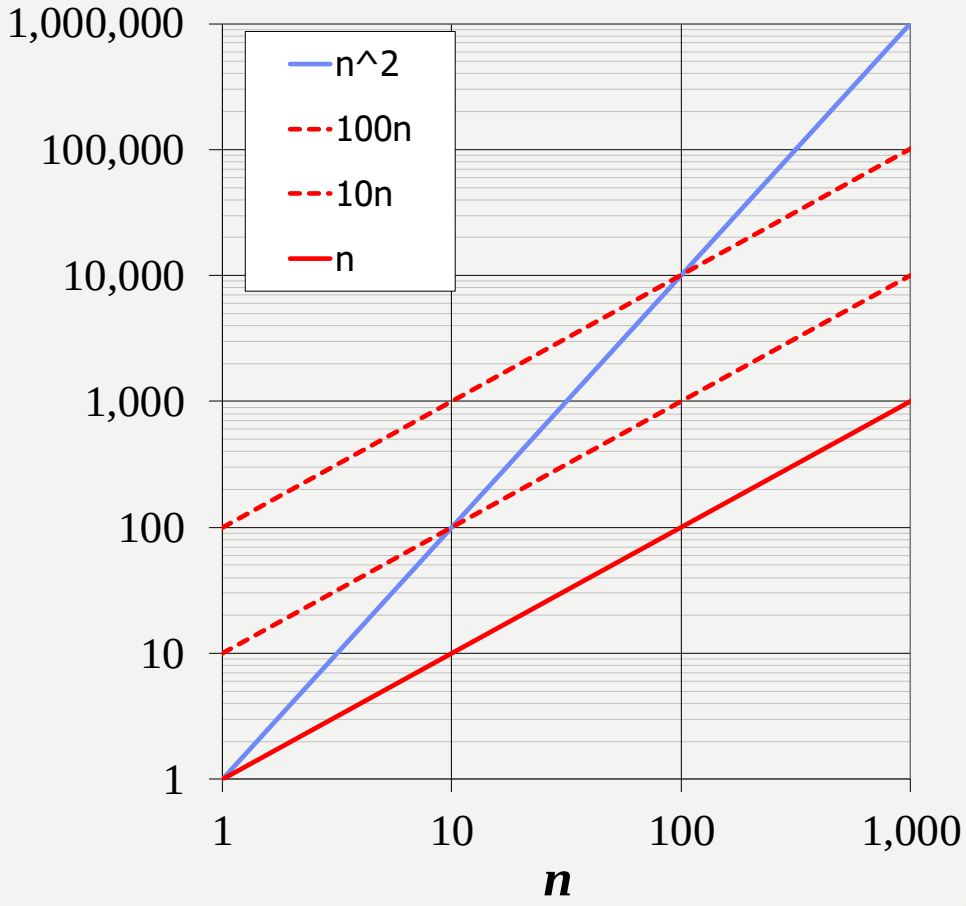


Big-O Example

- Example: the function n^2 is not $O(n)$
- Can you choose c and n_0
- Do it by yourself

Big-O Example

- Example: the function n^2 is not $O(n)$
  - $n^2 \leq cn$
  - $n \leq c$
  - The above inequality cannot be satisfied since c must be a constant
  - n^2 is $O(n^2)$.



More Big-O Examples

- $7n-2$
- $7n-2$ is $O(n)$
- need $c>0$ and $n_0 \geq 1$ such that $7n-2 \leq cn$ for $n \geq n_0$
- this is true for $c=7$ and $n_0=1$

More Big-O Examples

- $3n^3+20n^2+5$
- $3n^3+20n^2+5$ is $O(n^3)$
- need $c>0$ and $n_0 \geq 1$ such that $3n^3+20n^2+5 \leq cn^3$ for $n \geq n_0$
- this is true for $c = 4$ and $n_0 = 21$

More Big-O Examples

- $3 \log n + 5$
- $3 \log n + 5$ is $O(\log n)$
- need $c > 0$ and $n_0 \geq 1$ such that $3 \log n + 5 \leq c \log n$ for $n \geq n_0$
- this is true for $c = 8$ and $n_0 = 2$

More Big-O Examples

- $10000n + 5$
- $10000n$ is $O(n)$
- $f(n)=10000n$ and $g(n)=n$, $n_0 = 10000$ and $c = 1$ then $f(n) \leq 1*g(n)$ where $n \geq n_0$ and we say that $f(n) = O(g(n))$

More examples

- What about $f(n) = 4n^2$? Is it $O(n)$?
  - Find a c such that $4n^2 < cn$ for any $n > n_0$
  - $4n<c$ and thus c is not a constant.
- $50n^3 + 20n + 4$ is $O(n^3)$
  - Would be correct to say is $O(n^3+n)$
    - Not useful, as n^3 exceeds by far n, for large values
  - Would be correct to say is $O(n^5)$
    - OK, but $g(n)$ should be as closed as possible to $f(n)$
- $3\log(n) + \log(\log(n)) = O(\ ?\ )$

Properties of the O notation

- Constant factors may be ignored
  - $\forall\ k > 0,\ kf$ is $O(f)$

Properties of the O notation

- Constant factors may be ignored
  - $\forall\ k > 0,\ kf$ is $O(f)$
- Higher powers grow faster
  - n^r is $O(n^s)$ if $0 \le r \le s$

Properties of the O notation

- Constant factors may be ignored
  - $\forall\ k > 0,\ kf$ is $O(f)$
- Higher powers grow faster
  - n^r is $O(n^s)$ if $0 \le r \le s$

Fastest growing term dominates a sum

- If f is $O(g)$, then $f + g$ is $O(g)$

  $eg\quad an^4 + bn^3$ is $O(n^4)$

Properties of the O notation

- Constant factors may be ignored
  - $\forall\ k > 0,\ kf$ is $O(f)$
- Higher powers grow faster
  - n^r is $O(n^s)$ if $0 \le r \le s$

Fastest growing term dominates a sum

- If f is $O(g)$, then $f + g$ is $O(g)$

  eg $an^4 + bn^3$ is $O(n^4)$

Polynomial's growth rate is determined by leading term

- If f is a polynomial of degree d,

  then f is $O(n^d)$

Properties of the O notation

- f is $O(g)$ is transitive
  - If f is $O(g)$ and g is $O(h)$ then f is $O(h)$

Properties of the O notation

- f is $O(g)$ is transitive
  - If f is $O(g)$ and g is $O(h)$ then f is $O(h)$
- Product of upper bounds is upper bound for the product
  - If f is $O(g)$ and h is $O(r)$ then fh is $O(gr)$

Properties of the O notation

- f is $O(g)$ is transitive
  - If f is $O(g)$ and g is $O(h)$ then f is $O(h)$
- Product of upper bounds is upper bound for the product
  - If f is $O(g)$ and h is $O(r)$ then fh is $O(gr)$
- Exponential functions grow faster than powers
  - n^k is $O(b^n)$ $\forall$ $b > 1$ and $k \geq 0$

    eg n^{20} is $O(1.05^n)$

Properties of the O notation

- f is $O(g)$ is transitive
  - If f is $O(g)$ and g is $O(h)$ then f is $O(h)$
- Product of upper bounds is upper bound for the product
  - If f is $O(g)$ and h is $O(r)$ then fh is $O(gr)$
- Exponential functions grow faster than powers
  - n^k is $O(b^n)$ $\forall$ $b > 1$ and $k \geq 0$

    eg n^{20} is $O(1.05^n)$
- Logarithms grow more slowly than powers
  - $\log_b n$ is $O(n^k)$ $\forall$ $b > 1$ and $k > 0$

    eg $\log_2 n$ is $O(n^{0.5})$

Properties of the O notation

- f is $O(g)$ is transitive
  - If f is $O(g)$ and g is $O(h)$ then f is $O(h)$
- Product of upper bounds is upper bound for the product
  - If f is $O(g)$ and h is $O(r)$ then fh is $O(gr)$
- Exponential functions grow faster than powers
  - n^k is $O(b^n)$ $\forall$ $b > 1$ and $k \geq 0$

    eg n^{20} is $O(1.05^n)$
- Logarithms grow more slowly than powers
  - $\log_b n$ is $O(n^k)$ $\forall$ $b > 1$ and $k > 0$

    eg $\log_2 n$ is $O(n^{0.5})$

Important!

Properties of the O notation

- All logarithms grow at the same rate
  - $\log_b n$ is $O(\log_d n)$ $\forall$ $b, d > 1$

Properties of the O notation

- All logarithms grow at the same rate
  - $\log_b n$ is $O(\log_d n)$ $\forall$ $b, d > 1$

- Sum of first n r^{th} powers grows as the $(r+1)^{th}$ power
  - $\sum_{k=1}^{n} k^r$ is $\Theta(\ n^{r+1}\ )$

$$eg \quad \sum_{k=1}^{n} i = \frac{n(n+1)}{2} \quad \text{is} \quad \Theta(\ n^2\ )$$

Derivation

- Assume $S=1^2+2^2+....+n^2$
- $(n+1)^3-n^3 = 3n^2+3n+1$
- $n^3-(n-1)^3 = 3(n-1)^2+3(n-1)+1$
- ...
- ..
- ...
- $2^3-1^3 = 3*1^2+3*1+1$
- Sum up all n equations, we have:
- $(n+1)^3-1 = 3* [1^2+2^2+...+n^2] +3*[1+2+....+n] +n$
- We have $S= (1/3)*[(n+1)^3-1-n-(1/2)*n(n+1)] = (1/6)n(n+1)(2n+1)$

Polynomial and Intractable Algorithms

- Polynomial Time complexity
  - An algorithm is said to be polynomial if it is $O(n^d)$ for some integer d
  - Polynomial algorithms are said to be efficient
    - They solve problems in reasonable times!

- Intractable algorithms
  - Algorithms for which there is no *known* polynomial time algorithm
  - *We will come back to this important class later in the course*

Analysing an Algorithm

- Simple statement sequence

  $s_1;\ s_2;\ \ldots.\ ;\ s_k$

  – $O(1)$ as long as k is constant

- Simple loops

  ```
  for(i=0;i<n;i++) { s; }
  ```

  where `s` is $O(1)$

  – Time complexity is $n\,O(1)$ or $O(n)$

- Nested loops

  ```
  for(i=0;i<n;i++)
     for(j=0;j<n;j++) { s; }
  ```

  – Complexity is $n\,O(n)$ or $O(n^2)$

This part is $O(n)$

Analysing an Algorithm

- Loop index doesn’t vary linearly

```
h = 1;
while ( h <= n ) {
      s;
      h = 2 * h;
      }
```

Analysing an Algorithm

- Loop index doesn't vary linearly

```
h = 1;
while ( h <= n ) {
        s;
        h = 2 * h;
        }
```

  - `h` takes values $1, 2, 4, \ldots$ until it exceeds n
  - There are $1 + \log_2 n$ iterations
  - Complexity $O(\log n)$

Analysing an Algorithm

- Loop index depends on outer loop index

```
for(j=0;j<n;j++)
    for(k=0;k<j+1;k++){
      s;
      }
```

Analysing an Algorithm

- Loop index depends on outer loop index

```
for(j=0;j<n;j++)
    for(k=0;k<j+1;k++){
      s;
      }
```

  – Inner loop executed
    - 1, 2, 3, …., n times

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

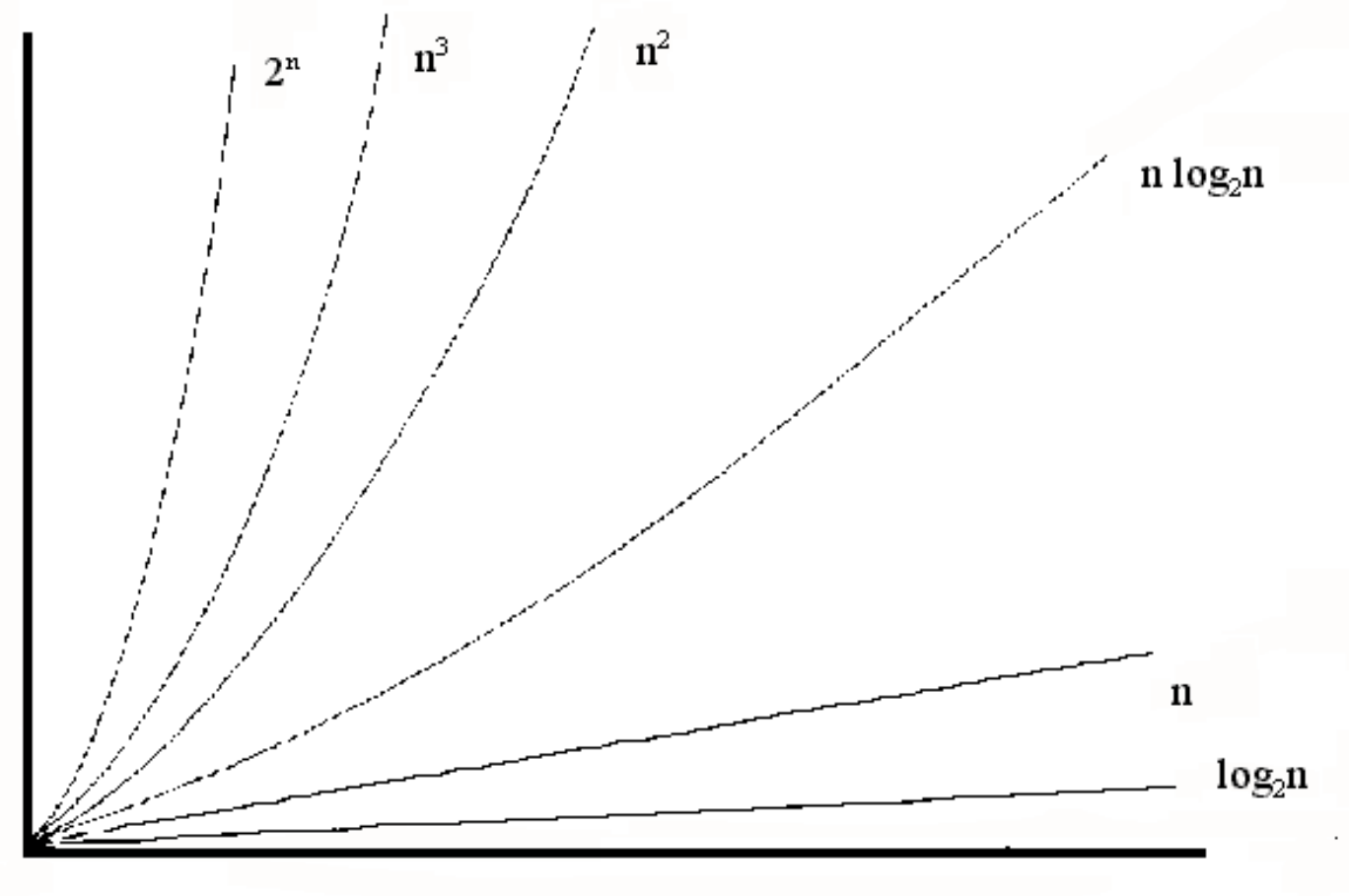
$\therefore$ Complexity $O(n^2)$

Distinguish this case - where the iteration count increases (decreases) by a constant $O(n^k)$ from the previous one - where it changes by a factor $O(\log n)$

The Growth Rate of the Six Popular functions

n	$\log n$	n	$n\log n$	n^2	n^3	2^n
4	2	4	8	16	64	16
8	3	8	24	64	512	256
16	4	16	64	256	4,096	65,536
32	5	32	160	1,024	32,768	4,294,967,296
64	6	64	384	4,094	262,144	$1.84 * 10^{19}$
128	7	128	896	16,384	2,097,152	$3.40 * 10^{38}$
256	8	256	2,048	65,536	16,777,216	$1.15 * 10^{77}$
512	9	512	4,608	262,144	134,217,728	$1.34 * 10^{154}$
1024	10	1,024	10,240	1,048,576	1,073,741,824	$1.79 * 10^{308}$

Common growth rates



Big-O Examples

Suppose a program P is $O(n^3)$, and a program Q is $O(3^n)$, and that currently both can solve problems of size 50 in 1 hour. If the programs are run on another system that executes exactly 729 times as fast as the original system, what size problems will they be able to solve?

Big-Oh Examples

$n^3 = 50^3 * 729$

$n = \sqrt[3]{50^3} * \sqrt[3]{729}$

$n = 50 * 9$

$n = 50 * 9 = 450$

$3^n = 3^{50} * 729$

$n = \log_3 (729 * 3^{50})$

$n = \log_3(729) + \log_3 3^{50}$

$n = 6 + \log_3 3^{50}$

$n = 6 + 50 = 56$

- Improvement: problem size increased by 9 times for n^3 algorithm but only a slight improvement in problem size (+6) for exponential algorithm.

Suggested Reading

- Chapter 10, Introduction to Algorithms, Thomas H. Cormen, 3rd Edition